

Day 15 Part 1 — DataLoader, Batching, Dropout & Save/Load

Dataset · DataLoader · Batches · Dropout · Save & Load

Target: Google · Meta · OpenAI · Anthropic

The Big Picture

Days 13 and 14 trained on all data at once — 400 samples loaded into memory together. Real ML datasets have millions of samples. You cannot load them all at once. Today you learn how PyTorch handles real data: loading in batches, shuffling, parallelising data loading, and preventing overfitting with Dropout. You also learn how to save and reload trained models.

File	What it does
day15_warmup_dataset.py	Practice: build a custom Dataset class from scratch
day15_warmup_batching.py	Practice: understand what batching does to training
day15_warmup_dropout.py	Practice: see dropout in action
day15_main.py	Full pipeline: Dataset, DataLoader, Dropout, save/load

Concept 1: Dataset — Teaching PyTorch About Your Data

A Dataset is a class that tells PyTorch two things: how many samples exist, and how to get one specific sample by index. That is all it does. Think of it like a library catalogue — it tells you how many books exist and where to find any specific book. You fetch them one at a time when needed.

Method	What it returns	Why PyTorch needs it
<code>__len__(self)</code>	Total number of samples as int	So PyTorch knows when one epoch is complete
<code>__getitem__(self, idx)</code>	One sample at index idx	So DataLoader can fetch any sample by position

Building a Dataset step by step:

Step 1 — Import and subclass Dataset:

```
import torch
from torch.utils.data import Dataset

class HiringDataset(Dataset):
    pass # we fill this in next
```

Step 2 — Add `__init__` to store data:

```
class HiringDataset(Dataset):
    def __init__(self, X: torch.Tensor, y: torch.Tensor) -> None:
        # Store features and labels as instance variables
        # self.X and self.y accessible in all other methods
        self.X = X # feature matrix – shape (n_samples, n_features)
        self.y = y # target vector – shape (n_samples, 1)
```

Step 3 — Add `__len__` to return dataset size:

```
def __len__(self) -> int:
    # Return total number of samples
    return len(self.X)
```

Step 4 — Add `__getitem__` to return one sample:

```
def __getitem__(self, idx: int):
    # PyTorch calls this with an index 0, 1, 2, ...
    # Return (features, label) tuple for that sample
    return self.X[idx], self.y[idx]
```

Using the Dataset:

```
dataset = HiringDataset(X_tensor, y_tensor)
print(f"Dataset size: {len(dataset)}") # calls __len__
sample_X, sample_y = dataset[0] # calls __getitem__(0)
print(f"Features: {sample_X}") # shape (4,)
print(f"Label: {sample_y}") # shape (1,)
```

Concept 2: DataLoader — Feeding Data to the Network

`DataLoader` wraps a `Dataset` and handles batching, shuffling, and parallel loading. Think of it as a conveyor belt in a factory — picks up groups of items (batches) and delivers them to the network in an organised stream.

Parameter	What it does	Typical value
<code>dataset</code>	Which Dataset to load from	Your Dataset instance
<code>batch_size</code>	How many samples per batch	32 or 64 for small data
<code>shuffle</code>	Randomise order each epoch	True for training, False for test
<code>num_workers</code>	CPU processes for loading	0 on Windows (safe default)
<code>drop_last</code>	Drop incomplete final batch	False usually

```
from torch.utils.data import DataLoader

# Training loader – shuffle=True randomises each epoch
train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32, # network sees 32 samples at a time
    shuffle=True, # different order every epoch
    num_workers=0 # 0 is safe on Windows
)

# Test loader – no shuffle needed
test_loader = DataLoader(
    dataset=test_dataset,
```

```

    batch_size=32,
    shuffle=False,
    num_workers=0
)

# Iterating through batches
for batch_X, batch_y in train_loader:
    # batch_X shape: (32, 4)
    # batch_y shape: (32, 1)
    print(f"Batch X: {batch_X.shape}")
    break

```

Concept 3: What Is a Batch and Why Does It Matter?

A batch is a small subset of training data. Instead of computing gradients on all 400 samples at once, you compute on 32 samples at a time. Mini-batch is the universal standard in production ML.

Approach	What it does	Pros	Cons
Full batch	Gradient on all data	Stable gradients	Cannot fit huge datasets in memory
Mini-batch (32-64)	Gradient on small batch	Memory efficient, stable	Slightly noisier gradients
Single sample	Gradient on 1 sample	Maximum noise	Very slow and unstable

With 400 samples and `batch_size=32`: $400/32 = 12.5 \rightarrow 13$ batches per epoch. Each epoch does 13 weight updates instead of 1. Training converges faster.

```

# The training loop with batching
for epoch in range(epochs):
    for batch_X, batch_y in train_loader:  # 13 iterations per epoch
        optimizer.zero_grad()
        output = model(batch_X)
        loss = criterion(output, batch_y)
        loss.backward()
        optimizer.step()
    # epoch done - all 13 batches processed

```

Concept 4: Dropout — Fighting Overfitting

Dropout randomly sets a fraction of neuron outputs to zero during each training step. This forces the network to learn redundant representations and prevents memorising training data. Think of it like a sports team that randomly loses players each game — everyone must be capable of covering for others. Result: a more robust team.

Mode	What happens
Training (model.train())	Each neuron randomly set to 0 with probability p. Remaining neurons scaled up by $1/(1-p)$.
Evaluation (model.eval())	All neurons active — no dropout. Deterministic output every time.

How to add Dropout to a network:

```
import torch.nn as nn

class HiringNetWithDropout(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.layer1 = nn.Linear(4, 8)
        self.layer2 = nn.Linear(8, 4)
        self.layer3 = nn.Linear(4, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
        # Dropout(p=0.3) – 30% of neurons randomly set to 0
        # Common values: 0.2 to 0.5
        # Higher p = more regularisation = less overfitting
        self.dropout = nn.Dropout(p=0.3)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.layer1(x))
        x = self.dropout(x)          # dropout after layer 1
        x = self.relu(self.layer2(x))
        x = self.dropout(x)          # dropout after layer 2
        x = self.sigmoid(self.layer3(x)) # NO dropout on output layer
        return x
```

CRITICAL: Always call `model.eval()` before evaluation. If dropout is still active during testing, neurons are randomly zeroed and you get different results every run.

```
model.train() # dropout active – use during training loop
model.eval()  # dropout disabled – use before evaluation
```

Concept 5: Saving and Loading Models

Training takes time. You need to save your trained model to disk so you can reload it later without retraining. PyTorch saves the model's learned weights as a state dict — a dictionary mapping each layer name to its weight tensors.

Approach	What is saved	When to use
<code>torch.save(model.state_dict(), path)</code>	Only weights and biases	Recommended — flexible, smaller file

<code>torch.save(model, path)</code>	Entire model including architecture	Convenient but less portable
--------------------------------------	-------------------------------------	------------------------------

Saving:

```
# Save after training is complete
torch.save(model.state_dict(), 'hiring_model.pth')
print("Model saved")

# Inspect what was saved
for name, param in model.state_dict().items():
    print(f"{name}: shape {param.shape}")
# layer1.weight: shape torch.Size([8, 4])
# layer1.bias:   shape torch.Size([8])
# etc.
```

Loading:

```
# Step 1: create model with same architecture (empty weights)
loaded_model = HiringNetWithDropout()

# Step 2: load saved weights
# map_location handles GPU/CPU differences
loaded_model.load_state_dict(
    torch.load('hiring_model.pth', map_location=torch.device('cpu'))
)

# Step 3: set to eval mode
loaded_model.eval()

# Verify – should give same predictions as original
with torch.no_grad():
    predictions = loaded_model(X_test)
    accuracy = (predictions > 0.5).float().eq(y_test).float().mean().item()
    print(f"Loaded model accuracy: {accuracy:.3f}")
```

Warm-Up 1 — day15_warmup_dataset.py

Build a Dataset and DataLoader from scratch. Verify they work before using in training.

```
import torch
from torch.utils.data import Dataset, DataLoader

class NumbersDataset(Dataset):
    # X = numbers 0-99, y = 1 if even, 0 if odd
    def __init__(self, size: int = 100) -> None:
        self.X = torch.arange(0, size, dtype=torch.float32).reshape(-1, 1)
        self.y = (self.X % 2 == 0).float()

    def __len__(self) -> int:
        return len(self.X)

    def __getitem__(self, idx: int):
        return self.X[idx], self.y[idx]

dataset = NumbersDataset(100)
print(f"Dataset size: {len(dataset)}")
x0, y0 = dataset[0]
x1, y1 = dataset[1]
print(f"Sample 0: X={x0.item()}, y={y0.item()} (0 is even -> 1.0)")
print(f"Sample 1: X={x1.item()}, y={y1.item()} (1 is odd -> 0.0)")
print()

# Wrap in DataLoader
loader = DataLoader(dataset, batch_size=10, shuffle=False)
print("First 3 batches:")
for i, (bX, by) in enumerate(loader):
    print(f"  Batch {i+1}: X shape={bX.shape}, y shape={by.shape}")
    print(f"  X values: {bX.squeeze().tolist()}")
    if i >= 2:
        break

print(f"\nTotal batches: {len(loader)} (expected: 10)")

# Shuffled loader
loader_s = DataLoader(dataset, batch_size=10, shuffle=True)
print("Shuffled first batch:")
for bX, by in loader_s:
    print(f"  {bX.squeeze().tolist()}")
    break
```

Warm-Up 2 — day15_warmup_batching.py

Compare full-batch vs mini-batch training on the same simple problem.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# TensorDataset wraps tensors directly – no custom class needed
# Use when data is already tensors and you need no custom logic

torch.manual_seed(42)
```

```

n = 200
X = torch.randn(n, 1)
y = 2 * X + 1 + 0.1 * torch.randn(n, 1) # learn y = 2x + 1

X_train, X_test = X[:160], X[160:]
y_train, y_test = y[:160], y[160:]
train_ds = TensorDataset(X_train, y_train)

def train_and_eval(batch_size: int, epochs: int = 100) -> float:
    model = nn.Linear(1, 1)
    criterion = nn.MSELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01)
    loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    for _ in range(epochs):
        for bx, by in loader:
            optimizer.zero_grad()
            loss = criterion(model(bx), by)
            loss.backward()
            optimizer.step()
    model.eval()
    with torch.no_grad():
        return criterion(model(X_test), y_test).item()

print("Effect of batch size on test loss (lower = better):")
for bs in [1, 8, 32, 160]:
    loss = train_and_eval(bs)
    label = "full batch" if bs == 160 else f"batch={bs}"
    print(f" {label:15s}: test loss = {loss:.4f}")

```

Warm-Up 3 — day15_warmup_dropout.py

See dropout in action. Observe different behaviour in train vs eval mode.

```

import torch
import torch.nn as nn

print("=== Dropout Behaviour ===")
dropout = nn.Dropout(p=0.5) # 50% of values set to zero
x = torch.ones(1, 10)
print(f"Input: {x}")

# Training mode – dropout active
dropout.train()
print(f"Train run 1: {dropout(x)}")
print(f"Train run 2: {dropout(x)}")
print(f"Train run 3: {dropout(x)}")
print("Notice: different neurons zeroed each run")
print()

# Eval mode – dropout disabled
dropout.eval()
print(f"Eval mode: {dropout(x)}")
print("Notice: all values pass through unchanged")
print()

# Scale compensation
print("=== Scale Compensation ===")

```

```
# During training, remaining neurons scaled UP by 1/(1-p)
# With p=0.5: survivors multiplied by 2.0
# This keeps expected value the same in train and eval
dropout.train()
runs = torch.stack([dropout(torch.ones(1, 1000)) for _ in range(100)])
print(f"Mean of 100 training runs: {runs.mean().item():.3f} (should be ~1.0)")
```


Main File — day15_main.py

Full pipeline: Dataset, DataLoader, Dropout, training loop, save and load.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split

# 1. DATASET CLASS
class HiringDataset(Dataset):
    def __init__(self, X: torch.Tensor, y: torch.Tensor) -> None:
        self.X = X
        self.y = y
    def __len__(self) -> int:
        return len(self.X)
    def __getitem__(self, idx: int):
        return self.X[idx], self.y[idx]

# 2. CREATE DATA
np.random.seed(42)
n = 500
age = np.random.randint(22, 60, n).astype(float)
income = np.random.randint(20000, 100000, n).astype(float)
education = np.random.randint(8, 22, n).astype(float)
experience = np.random.randint(0, 20, n).astype(float)
hired = (0.3 * education + 0.5 * experience
         + np.random.normal(0, 2, n) > 10).astype(float)

X_np = np.column_stack([age, income, education, experience])
y_np = hired.reshape(-1, 1)
X_np = (X_np - X_np.mean(axis=0)) / X_np.std(axis=0)

X_tr, X_te, y_tr, y_te = train_test_split(X_np, y_np, test_size=0.2, random_state=42)
X_train = torch.tensor(X_tr, dtype=torch.float32)
X_test = torch.tensor(X_te, dtype=torch.float32)
y_train = torch.tensor(y_tr, dtype=torch.float32)
y_test = torch.tensor(y_te, dtype=torch.float32)

# 3. DATALOADERS
train_loader = DataLoader(HiringDataset(X_train, y_train), batch_size=32, shuffle=True)
test_loader = DataLoader(HiringDataset(X_test, y_test), batch_size=32, shuffle=False)
print(f"Training batches per epoch: {len(train_loader)}")

# 4. NETWORK WITH DROPOUT
class HiringNet(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.layer1 = nn.Linear(4, 16)
        self.layer2 = nn.Linear(16, 8)
        self.layer3 = nn.Linear(8, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
        self.dropout = nn.Dropout(p=0.3)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.layer1(x))
        x = self.dropout(x)
        x = self.relu(self.layer2(x))
```

```

        x = self.dropout(x)
        x = self.sigmoid(self.layer3(x))
        return x

# 5. SETUP
torch.manual_seed(42)
model = HiringNet()
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
print(f"Parameters: {sum(p.numel() for p in model.parameters())}")

# 6. TRAINING LOOP
print("Training with mini-batches...")
for epoch in range(50):
    model.train()
    epoch_loss = 0.0
    for bX, by in train_loader:
        optimizer.zero_grad()
        loss = criterion(model(bX), by)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    if (epoch + 1) % 10 == 0:
        model.eval()
        correct = sum(
            (model(bX) > 0.5).float().eq(by).sum().item()
            for bX, by in test_loader
        )
        acc = correct / len(y_test)
        print(f" Epoch {epoch+1:3d} | Loss: {epoch_loss/len(train_loader):.4f} | Acc: {acc:.3f}")

# 7. FINAL EVALUATION
model.eval()
with torch.no_grad():
    correct = sum((model(bX)>0.5).float().eq(by).sum().item() for bX,by in test_loader)
final_acc = correct / len(y_test)
print(f"\nFinal accuracy: {final_acc:.3f} ({correct}/{len(y_test)})")

# 8. SAVE
torch.save(model.state_dict(), 'hiring_model.pth')
print("Model saved to hiring_model.pth")

# 9. LOAD AND VERIFY
loaded = HiringNet()
loaded.load_state_dict(torch.load('hiring_model.pth', map_location='cpu'))
loaded.eval()
with torch.no_grad():
    correct2 = sum((loaded(bX)>0.5).float().eq(by).sum().item() for bX,by in test_loader)
loaded_acc = correct2 / len(y_test)
print(f"Loaded model accuracy: {loaded_acc:.3f}")
print(f"Matches original: {abs(final_acc - loaded_acc) < 0.001}")

```

Expected Results

Metric	Expected	Notes
Training batches per epoch	13	400 samples / 32 = 12.5 -> 13

Loss at epoch 10	~0.55-0.65	Dropout adds noise — higher than Day 14
Loss at epoch 50	~0.40-0.50	Converging with regularisation
Final accuracy	~78-85%	Better generalisation due to dropout
Loaded model matches	True	Must match — confirms save/load works

Anki Cards — Day 15 Part 1

Q: What two methods must every Dataset implement?

A: `__len__()` returning total samples, `__getitem__(idx)` returning one sample

Q: What does DataLoader do?

A: Wraps a Dataset — handles batching, shuffling, and parallel loading

Q: What is a mini-batch?

A: A small subset of training data (32-64 samples) processed together

Q: Why `shuffle=True` in training DataLoader?

A: Prevents model learning data order — reduces bias and improves generalisation

Q: What does `nn.Dropout(p=0.3)` do during training?

A: Randomly sets 30% of neuron outputs to zero each forward pass

Q: What does `nn.Dropout` do during `model.eval()`?

A: Nothing — all neurons active, output is deterministic

Q: What is `state_dict`?

A: A dictionary of all model weights and biases — what gets saved to disk

Q: How do you save a model?

A: `torch.save(model.state_dict(), 'model.pth')`

Q: How do you load a model?

A: Create model with same architecture, then `model.load_state_dict(torch.load('model.pth'))`

Q: What is `TensorDataset`?

A: A shortcut Dataset class that wraps tensors directly — no custom class needed

Next: Day 15 Part 2 — Multi-class classification with Iris dataset and `CrossEntropyLoss`.